



LLM-Dienst der TU Darmstadt

Cheatsheet für den Workshop der AG Cyberraum

Alle Modellaufrufe im Workshop laufen über den LiteLLM-Dienst der TU Darmstadt. Der Dienst bietet eine OpenAI-kompatible API. Code für die OpenAI-API funktioniert daher mit geänderter Basisadresse und eurem Team-Key.

Eckdaten

Was	Wert
Basisadresse	<code>https://llm-service.ai.tu-darmstadt.de/v1</code>
Authentifizierung	Header <code>Authorization: Bearer <KEY></code>
Standardmodell	<code>GLM-5.2-AWQ-INT4</code>
Genutzte Routen	<code>GET /v1/models</code> , <code>POST /v1/chat/completions</code>
Key	pro Team, von den Lehrenden ausgegeben, beginnt mit <code>sk-</code>

Key eintragen

Slot	Datei	Variable
2 und 3, eigener Agent	<code>Agent/.env</code>	<code>LITELLM_API_KEY</code>
4, CAI	dieselbe <code>Agent/.env</code>	<code>LITELLM_API_KEY</code>

Im Verzeichnis Agent:

```
cp .env.example .env
```

Unter Windows PowerShell: `Copy-Item .env.example .env`. Tragt danach in `.env` euren Key ein:

```
LITELLM_API_KEY=euer-team-key  
LITELLM_MODEL=GLM-5.2-AWQ-INT4
```

Verbindung testen

macOS und Linux mit curl

Lädt den Key aus `.env`, ohne ihn in die Shell-History zu schreiben:

```
set -a; source .env; set +a
```

Verfügbare Modelle abfragen. In der Antwort muss `GLM-5.2-AWQ-INT4` vorkommen:

```
curl -s https://llm-service.ai.tu-darmstadt.de/v1/models \\\n-H "Authorization: Bearer $LITELLM_API_KEY"
```

Einen Chat-Aufruf senden. Die Antwort steht unter `choices[0].message.content`:

```
curl -s https://llm-service.ai.tu-darmstadt.de/v1/chat/completions \\\n-H "Authorization: Bearer $LITELLM_API_KEY" \\\n-H "Content-Type: application/json" \\\n-d '{"model": "GLM-5.2-AWQ-INT4",\n  "messages": [{"role": "user", "content": "Antworte nur mit OK."}]}'
```



Alle Systeme mit Python

Speichert den Code als `llm_check.py` im Verzeichnis `Agent` und führt ihn in der aktivierten virtuellen Umgebung mit `python llm_check.py` aus. Er nutzt dieselbe Modellroute wie der Agent:

```
import requests
from dotenv import load_dotenv
from llm_provider import resolve_model_route

load_dotenv()
route = resolve_model_route()
messages = [{"role": "user", "content": "Antworte nur mit OK."}]
response = requests.post(
    route.chat_completions_url, headers=route.headers,
    json={"model": route.model, "messages": messages}, timeout=60,
)
response.raise_for_status()
print(response.json()["choices"][0]["message"]["content"])
```

Slot 4: Modell-Gateway prüfen

CAI erreicht den TU-Dienst nur über den lokalen Modell-Gateway, der als einziger den Key kennt.

```
docker compose --env-file runtime/workshop.defaults.env \
  --env-file Agent/.env -f runtime/compose.yml \
  exec model-gateway curl --fail --silent http://127.0.0.1:8080/readyz
```

Nützliche Request-Parameter

- `"response_format": {"type": "json_object"}` erzwingt JSON, etwa für das Aktionsformat des Agenten.
- `"temperature": 0` macht Antworten gleichförmiger und Läufe besser vergleichbar.
- Ein Timeout von 60 Sekunden pro Aufruf reicht. Lange Antworten dauern mehrere Sekunden.

Fehlerbilder

Symptom	Ursache	Abhilfe
LITELLM_API_KEY fehlt	.env fehlt oder liegt im falschen Verzeichnis	.env in Agent anlegen und Key eintragen
HTTP 401, No api key passed in	Header fehlt oder Variable ist leer	.env laden, Header <code>Authorization: Bearer ...</code> prüfen
HTTP 401, Virtual Key expected	Platzhalter noch eingetragen oder <code>sk-</code> fehlt	vollständigen Key mit <code>sk-</code> eintragen
HTTP 401, Invalid proxy server token	Key falsch kopiert oder gesperrt	Key erneut eintragen, danach Lehrende fragen
HTTP 400 mit Hinweis auf das Modell	Modellname weicht ab	GLM-5.2-AWQ-INT4 exakt übernehmen
HTTP 429	Rate-Limit erreicht	kurz warten, nicht parallel starten
Timeout oder HTTP 5xx	Dienst ausgelastet oder gestört	wiederholen, dann Lehrende fragen

Regeln

- Der Key gehört nur in `Agent/.env`. Diese Datei ist von Git ignoriert und wird auch von Compose eingelesen.
- Kopiert den Key nicht in Prompts, Chats, Screenshots oder Commits.
- Sendet nur Workshopinhalte an das Modell, keine personenbezogenen oder vertraulichen Daten.
- Alle Teams teilen sich den Dienst. Lasst Schrittlimits aktiv und startet Läufe nicht mehrfach parallel.
- Wechselt nicht selbst den Anbieter. OpenRouter wird nur nach Ansage der Lehrenden aktiviert.
- Die Weboberfläche des Dienstes dient der Key-Verwaltung und erfordert eine TU-Anmeldung. Für den Workshop braucht ihr sie nicht.